Credit Name: CSE2130 - File Structures & Exception Handling

Assignment Name: Student Semester Average Mastery

How has your program changed from planning to coding to now? Please explain?

I started with making all the TextAreas, TextBoxes and Buttons I needed like the example photos. I then wanted to use a scroll pane as I didn't think that all text would be able to be display at once. I messed around with the properties of the scroll pane and added a TextArea so that text could be displayed.

```
JTextArea prompt = new JTextArea();
prompt.setEditable(false);
prompt.setFont(new Font("Calibri", Font.BOLD, 15));
prompt.setText("Student Name:\r\nGrade Level:\r\nSemester Number:\r\nGrade 1:\r\nGrade 2:\r\nGrade 3:\r\nGrade 4:");
prompt.setBounds(68, 10, 176, 144);
panel.add(prompt);

enterN = new JTextField();
enterN.setFont(new Font("Calibri", Font.PLAIN, 15));
enterN.setBackground(SystemColor.inactiveCaptionBorder);
enterN.setBounds(300, 10, 228, 20);
panel.add(enterN);
enterN.setColumns(10);

enterGrade = new JTextField();
enterGrade.setFont(new Font("Calibri", Font.PLAIN, 15));
enterGrade.setBackground(SystemColor.inactiveCaptionBorder);
enterGrade.setColumns(10);
enterGrade.setBounds(300, 30, 228, 20);
panel.add(enterGrade);

enterG2 = new JTextField();
enterG2.setFont(new Font("Calibri", Font.PLAIN, 15));
enterG2.setBackground(SystemColor.inactiveCaptionBorder);
enterG2.setColumns(10);
enterG2.setBounds(300, 90, 228, 20);
panel.add(enterG2);
```

```
enterSem = new JTextField();
enterSem.setFont(new Font("Calibri", Font.PLAIN, 15));
enterSem.setBackground(SystemColor.inactiveCaptionBorder);
enterSem.setColumns(10);
enterSem.setBounds(300, 50, 228, 20);
panel.add(enterSem);

enterG1 = new JTextField();
enterG1.setFont(new Font("Calibri", Font.PLAIN, 15));
enterG1.setBackground(SystemColor.inactiveCaptionBorder);
enterG1.setColumns(10);
enterG1.setBounds(300, 70, 228, 20);
panel.add(enterG1);

enterG3 = new JTextField();
enterG3.setFont(new Font("Calibri", Font.PLAIN, 15));
enterG3.setBackground(SystemColor.inactiveCaptionBorder);
enterG3.setColumns(10);
enterG3.setBounds(300, 110, 228, 20);
panel.add(enterG3);

enterG4 = new JTextField();
enterG4.setFont(new Font("Calibri", Font.PLAIN, 15));
enterG4.setBackground(SystemColor.inactiveCaptionBorder);
enterG4.setColumns(10);
enterG4.setBounds(300, 130, 228, 20);
panel.add(enterG4);
```

```
JLabel avgDis = new JLabel("Average: ");
avgDis.setFont(new Font("Calibri", Font.BOLD, 15));
avgDis.setBounds(69, 152, 228, 20);
panel.add(avgDis);

JButton saveFile = new JButton("Save To File");
saveFile.setVerticalAlignment(SwingConstants.TOP);
saveFile.setBackground(SystemColor.inactiveCaptionBorder);
saveFile.setFont(new Font("Calibri", Font.BOLD, 15));
saveFile.setForeground(new Color(0, 0, 0));
saveFile.setBounds(83, 296, 115, 23);
panel.add(saveFile);

JButton viewFile = new JButton("View File Contents");
viewFile.setVerticalAlignment(SwingConstants.TOP);
viewFile.setForeground(Color.BLACK);
viewFile.setFont(new Font("Calibri", Font.BOLD, 15));
viewFile.setBackground(SystemColor.inactiveCaptionBorder);
viewFile.setBounds(202, 296, 155, 23);
panel.add(viewFile);

scrollPane = new JScrollPane();
scrollPane.setHorizontalScrollBarPolicy(ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);
scrollPane.setVerticalScrollBarPolicy(ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS);
scrollPane.setBounds(37, 176, 514, 109);
panel.add(scrollPane);

displayInfo = new JTextArea();
displayInfo.setWrapStyleWord(true);
displayInfo.setEditable(false);
displayInfo.setLineWrap(true);
displayInfo.setBackground(SystemC
displayInfo.setFont(new Font("Cal
scrollPane.setViewportView(displa
```

```
JButton deleteFile = new JButton("Clear Entire File");
deleteFile.setVerticalAlignment(SwingConstants.TOP);
deleteFile.setForeground(Color.BLACK);
deleteFile.setFont(new Font("Calibri", Font.BOLD, 15));
deleteFile.setBackground(SystemColor.inactiveCaptionBorder);
deleteFile.setBounds(363, 296, 148, 23);
panel.add(deleteFile);
```

I created a File and DecimalFormat object as I am using calculations.

```
//Create file object
textFile = new File("../Chapter11/src/Mastery/GradeBook.txt");

//Shortens number to 2 decimal places
DecimalFormat df = new DecimalFormat("#0.00");
```

When the "Save File" button is clicked, the the data the user entered is assigned to varaibles. I use the data inputted to calculate an average grade and displaying it. I then created FileWriter and BufferedWriter objects and appened the string to the existing file by setting the boolean value of the FileWriter to true, before closing the objects. I then displayed that the action has been done. I used a method to reset all the boxes that the user typed in as a stylistic choice.

```
//Waits for user to click button
saveFile.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Get data that user inputed
        studentN = enterN.getText().toUpperCase();
        level = enterGrade.getText();
        sem = enterSem.getText();
        grd1 = Double.parseDouble(enterG1.getText());
        grd2 = Double.parseDouble(enterG2.getText());
        grd3 = Double.parseDouble(enterG3.getText());
        grd4 = Double.parseDouble(enterG4.getText());
```

```
//Method that resets the boxes that the user inputs data in
public void clearBox() {
    enterN.setText("");
    enterGrade.setText("");
    enterG2.setText("");
    enterSem.setText("");
    enterG1.setText("");
    enterG3.setText("");
    enterG4.setText("");
}
```

```
        grd4 = Double.parseDouble(enterGrd4.getText());
        }

        //Calculate and display average
        avg = (grd1 + grd2 + grd3 + grd4) /4;
        avgDis.setText("Average: " + avg);;

        //Tests code for errors
        try {
            //Create FileWriter and BufferedWriter object
            writes = new FileWriter(textFile, true);
            writeFile = new BufferedWriter(writes);

            //write data into file
            writeFile.write("Student Name: " + studentN + ", Grade Level: " + level + ", Semester Number: " + sem + ", Grade 1: " + grd1 +
                    ", Grade 2: " + grd2 + ", Grade 3: " + grd3 + ", Grade 4: " + grd4 + ", Average: " + df.format(avg) + "\n");

            //Close objects
            writeFile.close();
            writes.close();

            //Display confirmation of action
            displayInfo.setText("Graded Added!");

            //Reset boxes for user to input data
            clearBox();
        }
        catch (FileNotFoundException e1){
                System.out.println("File doesn't exist.");
                System.err.println("FileNotFOundException: " + e1.getMessage());
        }
        catch (IOException e1) {
            System.out.println("Problem reading file.");
            System.err.println("IOException: " + e1.getMessage());
        }
```

When the "View FIle" button is clicked, I used a method to reset all the boxes that the user typed in and created the FIleReader and BufferedReader object. I initialized string variables to display the text in the file. I check if the file was empty, if it was it was displayed to the user that it was empty. I closed and reopened the objects so that it started from the first line and added each line of the file to a string. I then displayed the entire string for the user and closed the objects. I wanted to add a display for when the file was empty after I already written the displaying part. I struggled, to integrate this ew part as the while loop would skip the first line. I realized that the if statement already read the first line making the while loop start at the second line, so I closed and reopened the object.

```
//Waits for user to click button
viewFile.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Reset boxes for user to input data
        clearBox();

        //Tests code for errors
        try {
            //Create FileReader and BufferedReader object
            reads = new FileReader(textFile);
            readFile = new BufferedReader(reads);

            //Initialize variables
            String eachLine, newLine = "";

            //Displays that file is empty if first line of file is empty
            if ((eachLine = readFile.readLine()) == null) {

                displayInfo.setText("Grade Book Is Empty.");
            }
            //Close objects
            readFile.close();
            reads.close();

            //Reinitialize objects so that first line is read
            reads = new FileReader(textFile);
            readFile = new BufferedReader(reads);

            //Continues loop while line in file isn't empty
            while ((eachLine = readFile.readLine()) != null) {
                //Adds each line to a string
                newLine = newLine + eachLine + "\n";
```

```
            //Displays string
            displayInfo.setText(newLine);
        }

        //Close objects
        readFile.close();
        reads.close();
    }
            System.out.println("File doesn't exist.");
            System.err.println("FileNotFOundException: " + e1.getMessage());
    }
    catch (IOException e1) {
        System.out.println("Problem reading file.");
        System.err.println("IOException: " + e1.getMessage());
    }
```

When the "Delete File" button is clicked, I created the Filewriter and BufferedWriter objects, but did not included *true* so that when I write "" in the file, it replaces any text in the file. I then displayed that the action has beendone and closing the objects.

```
//Waits for user to click button
deleteFile.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Tests code for errors
        try {
            //Create FileWriter and BufferedWriter object
            //FileWriter doesn't include true - replaces any text in file
            writes = new FileWriter(textFile);
            writeFile = new BufferedWriter(writes);

            //Replaces text in file with a blank
            writeFile.write("");

            //Displays that file has been cleared
            displayInfo.setText("Grade Book Cleared.");

            //Close objects
            writeFile.close();
            writes.close();
        }
        //Exception handling
        catch (FileNotFoundException e1){
            System.out.println("File doesn't exist.");
            System.err.println("FileNotFOundException: " + e1.getMessage());
        }
        catch (IOException e1) {
            System.out.println("Problem reading file.");
            System.err.println("IOException: " + e1.getMessage());
        }
    }
});
```